

```
In [28]: import numpy as np
import pandas as pd
import seaborn as sns
import xgboost as xgb
import lightgbm as lgb
import matplotlib.pyplot as plt

from catboost import CatBoostRegressor
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import warnings
warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
```

```
In [29]: data=pd.read_csv("C:/internship/weather data/merged_aqi.csv",parse_dates=["Datetime"])
print(f"Original data shape: {data.shape}")
print(f"Columns: {data.columns.tolist()}")
```

Original data shape: (707875, 24)
Columns: ['City', 'Datetime', 'PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI', 'AQI_Bucket', 'temperature', 'humidity', 'pressure', 'wind_speed', 'wind_direction', 'precipitation', 'cloud_cover', 'visibility']

```
In [30]: print(f"\nNumber of cities: {data['City'].unique()}")
print(f"\nDate range: {data['Datetime'].min()} to {data['Datetime'].max()}")
print(f"\nMissing AQI values: {data['AQI'].isnull().sum()}")
```

Number of cities: 26
Date range: 2015-01-01 01:00:00 to 2020-07-01 00:00:00
Missing AQI values: 129080

```
In [31]: print(f"\nShape before removing missing AQI: {data.shape}")
data = data.dropna(subset=["AQI"])
print(f"\nShape after removing missing AQI: {data.shape}")
```

Shape before removing missing AQI: (707875, 24)
Shape after removing missing AQI: (578795, 24)

```
In [32]: bins = [0, 50, 100, 150, 200, 300, 1500]
labels = ['Good', 'Moderate', 'USG', 'Unhealthy', 'Very Unhealthy', 'Hazardous']
data["AQI_Category"] = pd.cut(data["AQI"], bins=bins, labels=labels, right=True)
```

```
In [33]: pollutant_cols = ["PM2.5", "PM10", "NO2", "SO2", "CO", "O3", "NH3"]
weather_cols = ["temperature", "humidity", "pressure", "wind_speed",
               "wind_direction", "precipitation", "cloud_cover", "visibility"]

existing_pollutant_cols = [col for col in pollutant_cols if col in data.columns]
existing_weather_cols = [col for col in weather_cols if col in data.columns]

print(f"\nExisting pollutant columns: {existing_pollutant_cols}")
print(f"\nExisting weather columns: {existing_weather_cols}")

all_feature_cols = existing_pollutant_cols + existing_weather_cols
print(f"\nMissing values before imputation:")
```

```
missing_before = data[all_feature_cols].isnull().sum()
print(missing_before[missing_before > 0])
```

Existing pollutant columns: ['PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3', 'NH3']
Existing weather columns: ['temperature', 'humidity', 'pressure', 'wind_speed', 'wind_direction', 'precipitation', 'cloud_cover', 'visibility']

Missing values before imputation:

```
PM2.5      31035
PM10       178863
NO2         22267
SO2         33058
CO          29146
O3          33372
NH3        163994
dtype: int64
```

```
In [34]: print(f"\nApplying city-based median imputation...")
for col in all_feature_cols:
    if data[col].isnull().sum() > 0:
        print(f"    Imputing {col}: {data[col].isnull().sum()} missing values")
        # Use city-specific median, fallback to global median if city has no data
        data[col] = data.groupby('City')[col].transform(
            lambda x: x.fillna(x.median()) if x.median() != x.median() else x.fillna(x.median())
        )

print(f"\nMissing values after city-based imputation:")
missing_after = data[all_feature_cols].isnull().sum()
print(missing_after[missing_after > 0])

# Double-check: if any values are still missing, use global median as final fallback
remaining_missing = data[all_feature_cols].isnull().sum().sum()
if remaining_missing > 0:
    print(f"\nApplying global median for remaining {remaining_missing} missing values")
    imputer = SimpleImputer(strategy="median")
    data[all_feature_cols] = imputer.fit_transform(data[all_feature_cols])
    print("Final check - Missing values:", data[all_feature_cols].isnull().sum())
```

Applying city-based median imputation...

```
Imputing PM2.5: 31035 missing values
Imputing PM10: 178863 missing values
Imputing NO2: 22267 missing values
Imputing SO2: 33058 missing values
Imputing CO: 29146 missing values
Imputing O3: 33372 missing values
Imputing NH3: 163994 missing values
```

Missing values after city-based imputation:

```
Series([], dtype: int64)
```

```
In [35]: data["hour"] = data["Datetime"].dt.hour
data["dayofweek"] = data["Datetime"].dt.dayofweek
data["month"] = data["Datetime"].dt.month
data["year"] = data["Datetime"].dt.year
data["quarter"] = data["Datetime"].dt.quarter
```

```
In [36]: print("\nCreating lag features...")
lag_features = []
for lag in [1, 6, 12, 24]:
    lag_col = f"AQI_lag{lag}"
```

```
data[lag_col] = data.groupby("City")["AQI"].shift(lag)
lag_features.append(lag_col)
```

Creating lag features...

```
In [37]: for window in [3, 6, 12]:
        for col in ["AQI"] + existing_pollutant_cols[:3]: # Limit to avoid overfitt
            if col in data.columns:
                roll_col = f"{col}_rolling_{window}h"
                data[roll_col] = data.groupby("City")[col].rolling(window=window, mi
                if col == "AQI": # Only add AQI rolling features to lag_features
                    lag_features.append(roll_col)

        print(f"\nShape before removing missing lag features: {data.shape}")
        data = data.dropna(subset=lag_features)
        print(f"Shape after removing missing lag features: {data.shape}")
```

Shape before removing missing lag features: (578795, 46)

Shape after removing missing lag features: (578171, 46)

```
In [38]: q99 = data["AQI"].quantile(0.99)
        q01 = data["AQI"].quantile(0.01)
        print(f"\nRemoving AQI outliers beyond {q01:.1f} - {q99:.1f}")
        data = data[(data["AQI"] >= q01) & (data["AQI"] <= q99)]
        print(f"Shape after outlier removal: {data.shape}")

        # FIXED: Proper time series split - use chronological order
        print("\n" + "="*50)
        print("CREATING TIME SERIES TRAIN-TEST SPLIT")
        print("="*50)
```

Removing AQI outliers beyond 30.0 - 620.0

Shape after outlier removal: (567173, 46)

```
=====
CREATING TIME SERIES TRAIN-TEST SPLIT
=====
```

```
In [39]: data = data.sort_values("Datetime").reset_index(drop=True)

        # Use 80% for training, 20% for testing (chronologically)
        split_idx = int(0.8 * len(data))
        train_data = data.iloc[:split_idx].copy()
        test_data = data.iloc[split_idx:].copy()

        print(f"Training period: {train_data['Datetime'].min()} to {train_data['Datetime'].max()}")
        print(f"Testing period: {test_data['Datetime'].min()} to {test_data['Datetime'].max()}")
        print(f"Training set size: {train_data.shape}")
        print(f"Test set size: {test_data.shape}")
```

Training period: 2015-01-02 16:00:00 to 2019-12-05 19:00:00

Testing period: 2019-12-05 19:00:00 to 2020-07-01 00:00:00

Training set size: (453738, 46)

Test set size: (113435, 46)

```
In [40]: feature_columns = (existing_pollutant_cols + existing_weather_cols +
                            ["hour", "dayofweek", "month", "year", "quarter"] + lag_features)

        # Remove any columns that don't exist
        feature_columns = [col for col in feature_columns if col in data.columns]
        print(f"\nFinal feature columns ({len(feature_columns)}): {feature_columns}")
```

```

X_train = train_data[feature_columns]
y_train = train_data["AQI"]
X_test = test_data[feature_columns]
y_test = test_data["AQI"]

print(f"\nX_train shape: {X_train.shape}")
print(f"\nX_test shape: {X_test.shape}")

print(f"\nMissing values in training features: {X_train.isnull().sum().sum()}")
print(f"\nMissing values in test features: {X_test.isnull().sum().sum()}")

```

Final feature columns (27): ['PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3', 'NH3', 'temperature', 'humidity', 'pressure', 'wind_speed', 'wind_direction', 'precipitation', 'cloud_cover', 'visibility', 'hour', 'dayofweek', 'month', 'year', 'quarter', 'AQI_lag1', 'AQI_lag6', 'AQI_lag12', 'AQI_lag24', 'AQI_rolling_3h', 'AQI_rolling_6h', 'AQI_rolling_12h']

X_train shape: (453738, 27)

X_test shape: (113435, 27)

Missing values in training features: 0

Missing values in test features: 0

```

In [41]: print("\n" + "="*50)
print("TRAINING MODELS")
print("="*50)

models = {}
predictions = {}

# 1. CatBoost - handles categorical features well
print("\n1. Training CatBoost...")
catboost_model = CatBoostRegressor(
    iterations=300,
    learning_rate=0.1,
    depth=8,
    l2_leaf_reg=3,
    loss_function='RMSE',
    eval_metric='RMSE',
    verbose=50,
    random_seed=42
)
catboost_model.fit(
    X_train, y_train,
    eval_set=(X_test, y_test),
    early_stopping_rounds=30,
    use_best_model=True
)
models['CatBoost'] = catboost_model
predictions['CatBoost'] = catboost_model.predict(X_test)

```

```
=====
TRAINING MODELS
=====
```

1. Training CatBoost...

```
0:      learn: 101.8785451      test: 82.5494032      best: 82.5494032 (0)      t
otal: 41.7ms      remaining: 12.5s
50:      learn: 10.5253075      test: 6.4706824 best: 6.4706824 (50)      total: 2.
19s      remaining: 10.7s
100:     learn: 8.7652254      test: 5.5613512 best: 5.5613512 (100)     total: 4.
94s      remaining: 9.73s
150:     learn: 7.8506743      test: 5.1758534 best: 5.1758534 (150)     total: 7.
53s      remaining: 7.43s
200:     learn: 7.1876148      test: 4.9168739 best: 4.9168739 (200)     total: 1
0.2s     remaining: 5.02s
250:     learn: 6.7435513      test: 4.7387418 best: 4.7387418 (250)     total: 1
2.9s     remaining: 2.52s
299:     learn: 6.4107656      test: 4.6115065 best: 4.6115065 (299)     total: 16
s        remaining: 0us
```

```
bestTest = 4.611506547
bestIteration = 299
```

```
In [42]: print("\n2. Training XGBoost...")
xgb_model = xgb.XGBRegressor(
    n_estimators=300,
    learning_rate=0.1,
    max_depth=8,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.1,
    reg_lambda=1,
    random_state=42
)
xgb_model.fit(X_train, y_train)
models['XGBoost'] = xgb_model
predictions['XGBoost'] = xgb_model.predict(X_test)
```

2. Training XGBoost...

```
In [43]: print("\n3. Training LightGBM...")
lgb_model = lgb.LGBMRegressor(
    n_estimators=300,
    learning_rate=0.1,
    max_depth=8,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.1,
    reg_lambda=1,
    random_state=42,
    verbose=-1
)
lgb_model.fit(X_train, y_train, eval_set=[(X_test, y_test)],
              eval_metric='rmse',
              callbacks=[lgb.early_stopping(30)])
models['LightGBM'] = lgb_model
predictions['LightGBM'] = lgb_model.predict(X_test)
```

3. Training LightGBM...

Training until validation scores don't improve for 30 rounds

Did not meet early stopping. Best iteration is:

[299] valid_0's rmse: 4.3693 valid_0's l2: 19.0908

```
In [44]: print("\n4. Training Random Forest...")
rf_model = RandomForestRegressor(
    n_estimators=200,
    max_depth=15,
    min_samples_split=5,
    min_samples_leaf=2,
    max_features='sqrt',
    n_jobs=-1,
    random_state=42
)
rf_model.fit(X_train, y_train)
models['Random Forest'] = rf_model
predictions['Random Forest'] = rf_model.predict(X_test)
```

4. Training Random Forest...

```
In [45]: print("\n" + "="*50)
print("MODEL EVALUATION")
print("="*50)

results = []
for name, pred in predictions.items():
    mae = mean_absolute_error(y_test, pred)
    rmse = np.sqrt(mean_squared_error(y_test, pred))
    r2 = r2_score(y_test, pred)

    # Additional metrics
    mape = np.mean(np.abs((y_test - pred) / y_test)) * 100

    results.append({
        'Model': name,
        'MAE': mae,
        'RMSE': rmse,
        'R²': r2,
        'MAPE': mape
    })

print(f"\n{name}:")
print(f" MAE: {mae:.2f}")
print(f" RMSE: {rmse:.2f}")
print(f" R²: {r2:.3f}")
print(f" MAPE: {mape:.2f}%")
```

```
=====
MODEL EVALUATION
=====
```

CatBoost:

MAE: 2.09
RMSE: 4.61
R²: 0.997
MAPE: 1.97%

XGBoost:

MAE: 1.74
RMSE: 4.51
R²: 0.997
MAPE: 1.61%

LightGBM:

MAE: 1.92
RMSE: 4.37
R²: 0.997
MAPE: 1.82%

Random Forest:

MAE: 2.03
RMSE: 5.20
R²: 0.996
MAPE: 1.89%

```
In [46]: results_df = pd.DataFrame(results)
print(f"\n{'='*60}")
print("MODEL COMPARISON SUMMARY")
print('='*60)
print(results_df.round(3))
```

```
=====
MODEL COMPARISON SUMMARY
=====
```

	Model	MAE	RMSE	R ²	MAPE
0	CatBoost	2.088	4.612	0.997	1.967
1	XGBoost	1.744	4.510	0.997	1.613
2	LightGBM	1.919	4.369	0.997	1.817
3	Random Forest	2.030	5.205	0.996	1.888

```
In [47]: print(f"\n{'='*50}")
print("FEATURE IMPORTANCE ANALYSIS")
print('='*50)

if hasattr(catboost_model, 'get_feature_importance'):
    importance_df = pd.DataFrame({
        'Feature': feature_columns,
        'Importance': catboost_model.get_feature_importance()
    }).sort_values('Importance', ascending=False)

    print(f"\nTop 15 Most Important Features (CatBoost):")
    print(importance_df.head(15))

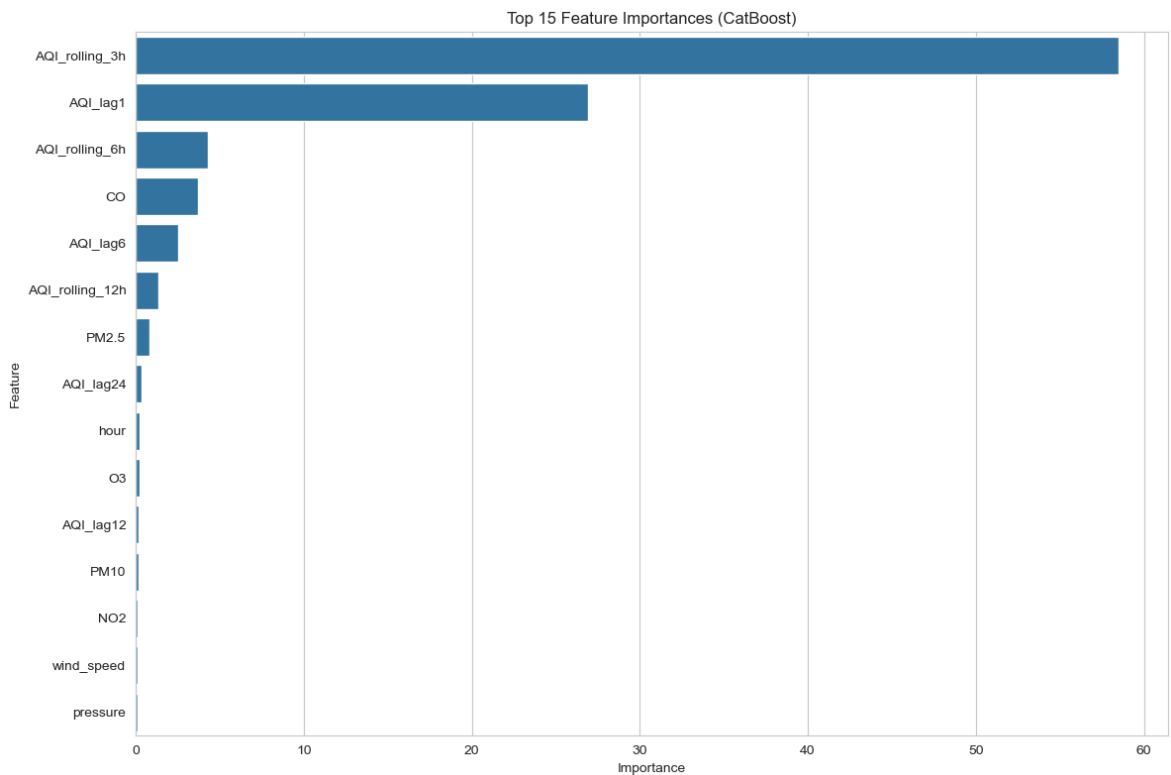
    # Plot feature importance
    plt.figure(figsize=(12, 8))
    sns.barplot(data=importance_df.head(15), y='Feature', x='Importance')
    plt.title('Top 15 Feature Importances (CatBoost)')
```

```
plt.tight_layout()
plt.show()
```

FEATURE IMPORTANCE ANALYSIS

Top 15 Most Important Features (CatBoost):

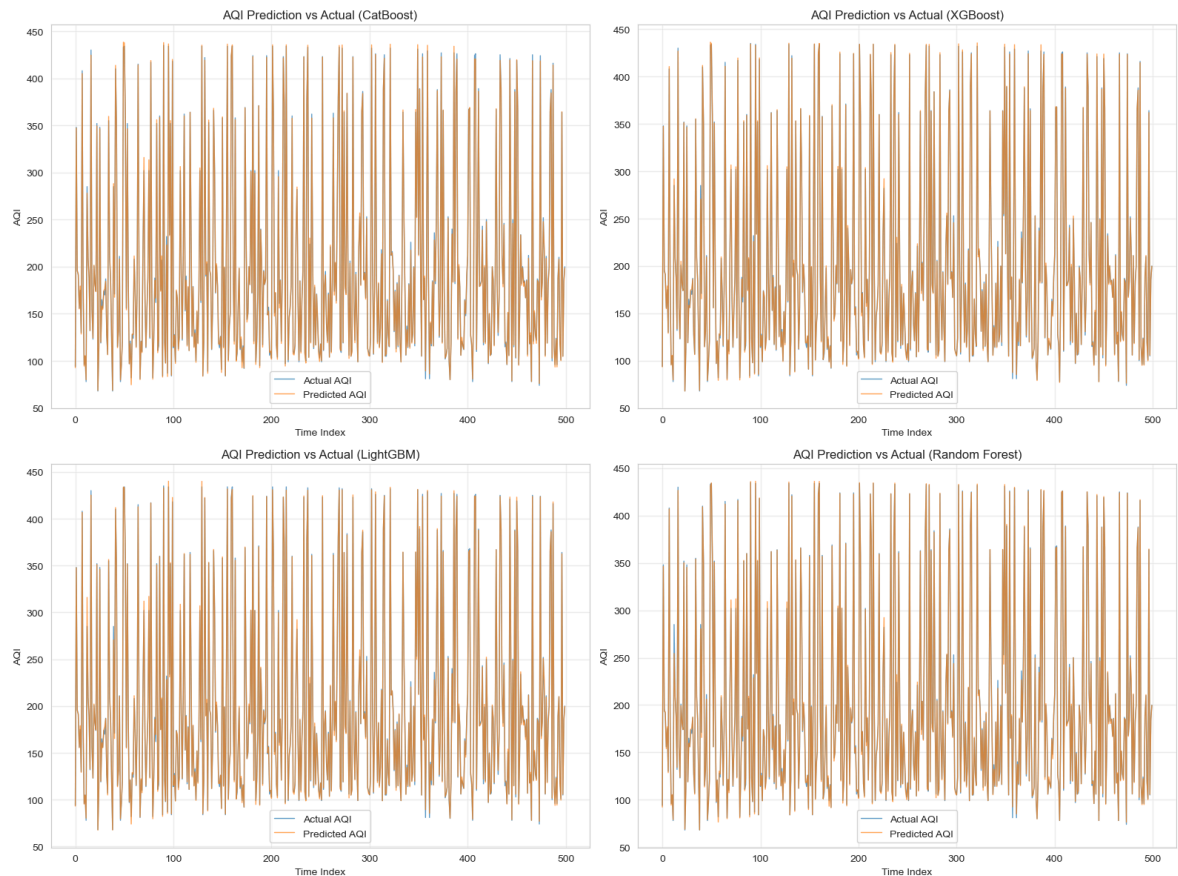
	Feature	Importance
24	AQI_rolling_3h	58.517111
20	AQI_lag1	26.896745
25	AQI_rolling_6h	4.295256
4	CO	3.710327
21	AQI_lag6	2.540458
26	AQI_rolling_12h	1.345137
0	PM2.5	0.841790
23	AQI_lag24	0.331685
15	hour	0.235338
5	O3	0.223414
22	AQI_lag12	0.178840
1	PM10	0.163777
2	NO2	0.130160
10	wind_speed	0.114104
9	pressure	0.085465



```
In [48]: fig, axes = plt.subplots(2, 2, figsize=(16, 12))
axes = axes.ravel()

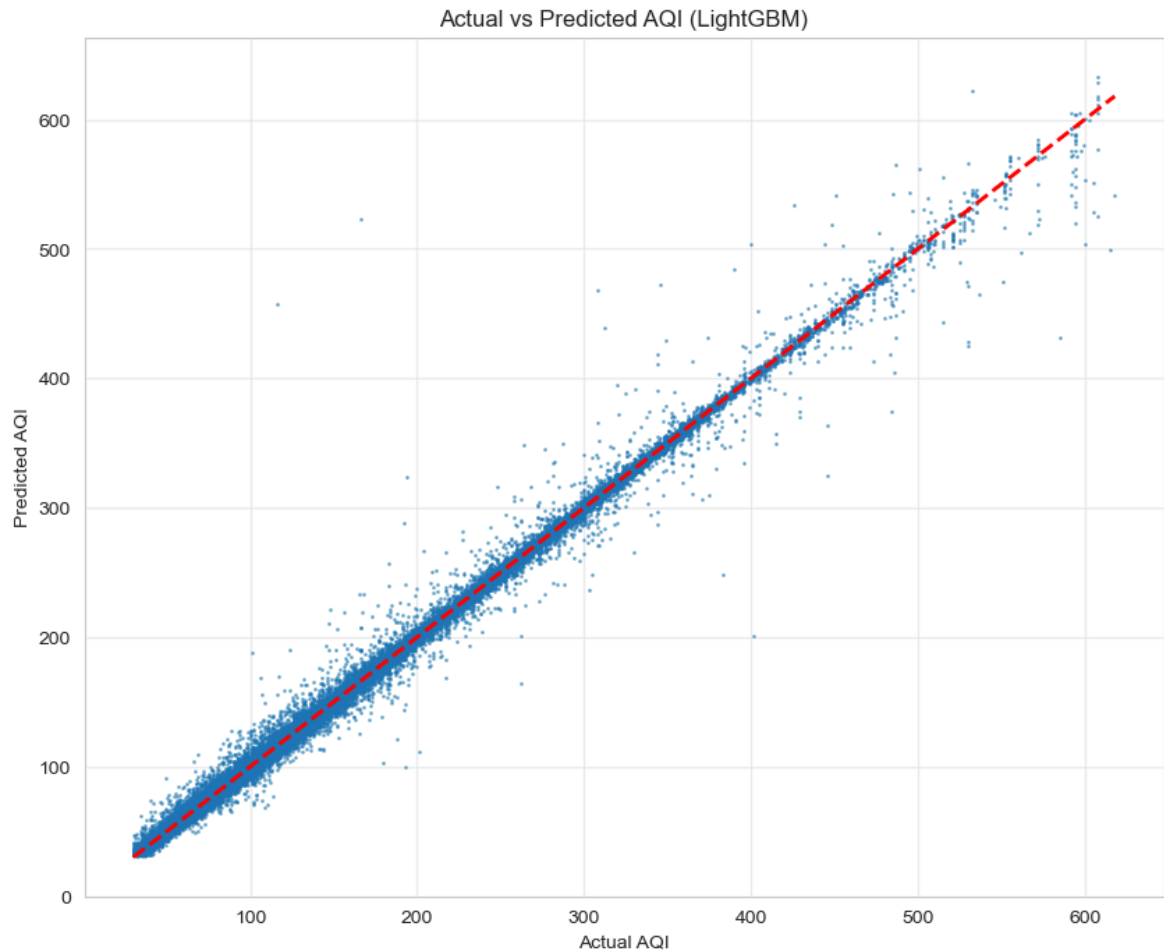
for i, (name, pred) in enumerate(predictions.items()):
    # Time series plot (first 500 points)
    axes[i].plot(range(500), y_test.iloc[:500].values, label="Actual AQI", alpha=0.7)
    axes[i].plot(range(500), pred[:500], label="Predicted AQI", alpha=0.7, linewidth=2)
    axes[i].set_title(f"AQI Prediction vs Actual ({name})")
    axes[i].set_xlabel("Time Index")
    axes[i].set_ylabel("AQI")
    axes[i].legend()
    axes[i].grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```



```
In [49]: best_model_name = results_df.loc[results_df['R²'].idxmax(), 'Model']
best_predictions = predictions[best_model_name]

plt.figure(figsize=(10, 8))
plt.scatter(y_test, best_predictions, alpha=0.5, s=1)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel('Actual AQI')
plt.ylabel('Predicted AQI')
plt.title(f'Actual vs Predicted AQI ({best_model_name})')
plt.grid(True, alpha=0.3)
plt.show()
```

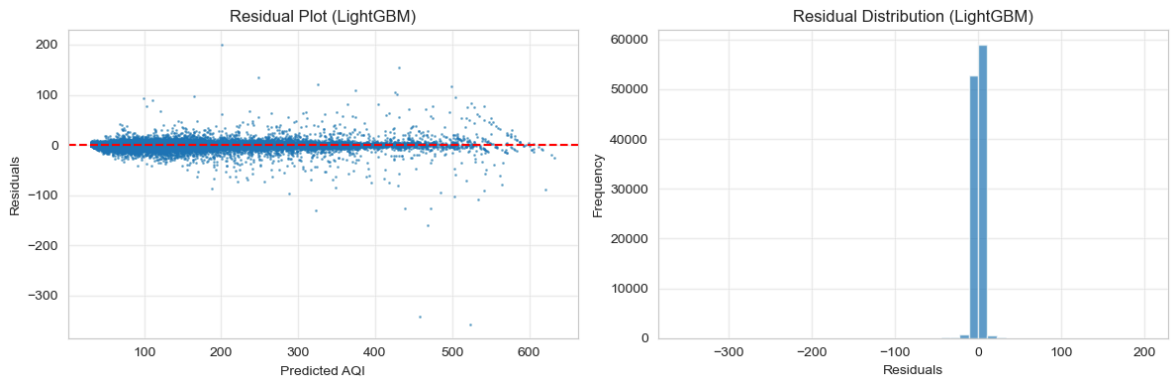


```
In [50]: residuals = y_test - best_predictions
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.scatter(best_predictions, residuals, alpha=0.5, s=1)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('Predicted AQI')
plt.ylabel('Residuals')
plt.title(f'Residual Plot ({best_model_name})')
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.hist(residuals, bins=50, alpha=0.7)
plt.xlabel('Residuals')
plt.ylabel('Frequency')
plt.title(f'Residual Distribution ({best_model_name})')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



```
In [51]: print(f"\n{'='*60}")
print("ANALYSIS COMPLETE")
print(f"{'='*60}")
print(f"Best performing model: {best_model_name}")
print(f"Best R2 score: {results_df['R2'].max():.3f}")
print(f"Best RMSE: {results_df.loc[results_df['R2'].idxmax(), 'RMSE']:.2f}")

# Save results for future use
print(f"\nSaving results...")
results_df.to_csv('model_comparison_results.csv', index=False)
print("Results saved to 'model_comparison_results.csv'")
```

```
=====
ANALYSIS COMPLETE
=====
```

```
Best performing model: LightGBM
Best R2 score: 0.997
Best RMSE: 4.37
```

```
Saving results...
```

```
Results saved to 'model_comparison_results.csv'
```

```
In [52]: # Add this code after your model training and predictions
# Assuming you have your best model (LightGBM) and test predictions

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Get your best model predictions (replace with your actual best model)
best_predictions = predictions['LightGBM'] # or whichever performed best

# Create analysis dataframe
analysis_df = test_data[['City', 'Datetime', 'AQI']].copy()
analysis_df['Predicted_AQI'] = best_predictions
analysis_df['Absolute_Error'] = np.abs(analysis_df['AQI'] - analysis_df['Predicted_AQI'])
analysis_df['Squared_Error'] = (analysis_df['AQI'] - analysis_df['Predicted_AQI'])**2
analysis_df['Percentage_Error'] = np.abs((analysis_df['AQI'] - analysis_df['Predicted_AQI']) / analysis_df['AQI'])

print(f"{'='*60}")
print("PER-CITY PERFORMANCE ANALYSIS")
print(f"{'='*60}")

# 1. Overall per-city metrics
city_metrics = []
cities = analysis_df['City'].unique()
```

```

for city in cities:
    city_data = analysis_df[analysis_df['City'] == city]

    if len(city_data) > 10: # Only analyze cities with sufficient data
        mae = mean_absolute_error(city_data['AQI'], city_data['Predicted_AQI'])
        rmse = np.sqrt(mean_squared_error(city_data['AQI'], city_data['Predicted_AQI']))
        r2 = r2_score(city_data['AQI'], city_data['Predicted_AQI'])
        mape = city_data['Percentage_Error'].mean()

        city_metrics.append({
            'City': city,
            'Count': len(city_data),
            'MAE': mae,
            'RMSE': rmse,
            'R2': r2,
            'MAPE': mape,
            'Mean_AQI': city_data['AQI'].mean(),
            'Std_AQI': city_data['AQI'].std(),
            'Max_Error': city_data['Absolute_Error'].max(),
            'Median_Error': city_data['Absolute_Error'].median()
        })

city_performance = pd.DataFrame(city_metrics).sort_values('RMSE', ascending=False)

print(f"\nPerformance by City (sorted by RMSE - worst first):")
print("="*100)
print(city_performance.round(3))

```

=====

PER-CITY PERFORMANCE ANALYSIS

=====

Performance by City (sorted by RMSE - worst first):

	City	Count	MAE	RMSE	R2	MAPE	Mean_AQI	Std_AQI	\
10	Ahmedabad	4290	7.699	17.053	0.981	4.164	192.208	123.262	
2	Talcher	3968	2.855	4.713	0.998	2.156	176.401	96.299	
15	Jorapokhar	4325	2.184	4.538	0.992	1.434	153.061	49.498	
18	Brajrajnagar	3188	2.315	4.362	0.993	1.819	139.063	53.466	
1	Gurugram	4995	2.460	3.745	0.998	1.728	176.057	86.454	
5	Guwahati	4837	2.313	3.637	0.999	1.893	180.725	116.961	
7	Patna	4954	1.919	3.443	0.999	1.258	185.625	93.600	
8	Chennai	4997	1.960	3.106	0.986	2.505	80.543	26.150	
14	Amritsar	4591	1.408	2.841	0.997	1.468	98.434	54.237	
19	Lucknow	4997	1.810	2.824	0.999	1.256	175.019	95.866	
12	Bhopal	4696	1.832	2.802	0.997	1.721	124.251	50.905	
24	Ernakulam	3328	1.482	2.781	0.981	1.709	92.357	20.221	
3	Kolkata	4979	1.787	2.745	0.999	1.976	129.306	81.585	
17	Thiruvananthapuram	4918	1.645	2.709	0.986	2.468	70.271	22.940	
22	Shillong	2628	1.484	2.636	0.992	2.540	68.147	29.841	
9	Coimbatore	3884	1.479	2.484	0.987	2.176	70.603	21.824	
4	Visakhapatnam	4604	1.455	2.394	0.997	1.806	90.047	41.878	
13	Delhi	4997	1.742	2.376	1.000	1.071	204.788	109.589	
6	Jaipur	4997	1.563	2.351	0.994	1.527	108.555	29.261	
0	Bengaluru	4998	1.541	2.242	0.988	2.017	80.272	20.292	
11	Chandigarh	4814	1.299	2.128	0.997	1.871	81.082	40.686	
23	Kochi	3694	1.225	1.918	0.997	1.159	104.671	35.522	
21	Amaravati	4733	1.172	1.913	0.997	2.021	68.460	34.727	
20	Mumbai	4997	1.137	1.638	0.999	1.111	114.618	59.910	
16	Hyderabad	4946	0.886	1.382	0.998	1.196	85.837	32.432	
25	Aizawl	1080	0.735	0.948	0.997	1.501	55.161	16.611	

	Max_Error	Median_Error
10	356.992	3.050
2	48.483	1.729
15	97.421	1.146
18	77.260	1.329
1	34.470	1.627
5	53.722	1.472
7	66.407	1.260
8	34.884	1.216
14	46.536	0.814
19	32.109	1.245
12	33.479	1.230
24	38.865	0.807
3	22.123	1.127
17	40.619	0.928
22	29.426	0.903
9	26.776	0.955
4	25.611	0.971
13	12.721	1.309
6	31.176	1.078
0	16.809	1.024
11	25.152	0.836
23	36.159	0.936
21	26.077	0.737
20	14.424	0.802

16	15.679	0.634
25	5.552	0.627

```
In [53]: # 2. Identify problematic cities
print(f"\n{'='*60}")
print("PROBLEM CITIES IDENTIFICATION")
print(f"{'='*60}")

# Define thresholds for "poor" performance
rmse_threshold = city_performance['RMSE'].quantile(0.75) # Top 25% worst RMSE
r2_threshold = city_performance['R2'].quantile(0.25)     # Bottom 25% R2

problem_cities = city_performance[
    (city_performance['RMSE'] > rmse_threshold) |
    (city_performance['R2'] < r2_threshold)
]

print(f"\nCities with Poor Performance (RMSE > {rmse_threshold:.2f} or R² < {r2_
if len(problem_cities) > 0:
    print(problem_cities[['City', 'Count', 'MAE', 'RMSE', 'R2', 'MAPE']].round(3
else:
    print("No cities with significantly poor performance detected!")
```

```
=====
PROBLEM CITIES IDENTIFICATION
=====
```

```
Cities with Poor Performance (RMSE > 3.36 or R² < 0.992):
```

	City	Count	MAE	RMSE	R2	MAPE
10	Ahmedabad	4290	7.699	17.053	0.981	4.164
2	Talcher	3968	2.855	4.713	0.998	2.156
15	Jorapokhar	4325	2.184	4.538	0.992	1.434
18	Brajrajnagar	3188	2.315	4.362	0.993	1.819
1	Gurugram	4995	2.460	3.745	0.998	1.728
5	Guwahati	4837	2.313	3.637	0.999	1.893
7	Patna	4954	1.919	3.443	0.999	1.258
8	Chennai	4997	1.960	3.106	0.986	2.505
24	Ernakulam	3328	1.482	2.781	0.981	1.709
17	Thiruvananthapuram	4918	1.645	2.709	0.986	2.468
9	Coimbatore	3884	1.479	2.484	0.987	2.176
0	Bengaluru	4998	1.541	2.242	0.988	2.017

In []: